Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Ödev sunumları: üç kelimeden cümle üretimi
  - Ödevde verilen üç kelimeyi içeren bir cümle üretmek isteniyordu. Sunumlarda Markov zinciri (kelime bigram/trigram), kelime ve karakter tabanlı LSTM, fastText vektörleri ve T5 denendi.
  - Bir grup, bir roman derlemi üzerinde Markov zinciriyle 1-gram (yalnız önceki kelimeye bakan) ve 2-gram (son iki kelimeye bakan) üretimi karşılaştırdı. Rastgele seçilen üç kelimenin ilkiyle zincir başlatıldı, en çok 200 kelimelik metin üretildi ve diğer iki kelimenin içinde geçip geçmediği kontrol edildi. 2-gram daha anlamlı cümleler üretti; 1-gram bazı üçlülerde 200 kelime içinde üç kelimeyi bir araya getiremedi.
  - Grubun slaytındaki sayılar (ekrandan, 00:10:00): tek derlemde toplam token 56.000, farklı token 15.000; üç roman birleştirilince toplam token 435.000, farklı token 78.000 ve üretim süresi çok uzadı. 20 kelime sınırı konunca da üretim çok uzun sürdü, cümleler çoğunlukla anlamsızdı.
  - Örnek (ekrandan, 00:10:00): özgün "Körle kötürümün yoldaşlığı gibi bir iş için değil!" cümlesinden 2-gram modeli "Körle kötürümün yoldaşlığı gibi bir gerçek vardı" cümlesini üretti.
  - Bir öğrenci kelime tabanlı Markov zinciri ile karakter tabanlı LSTM'i karşılaştırdı. Verilen kelimeleri araya kendisi ekleyip üretime devam ettirdi. Markov'un ikili-üçlü kelime grupları daha anlamlıydı; karakter LSTM bazen var olmayan kelimeler üretti.
  - Bir grup film yorumları derleminden (1500 yorum) Markov zinciri ve kelime tabanlı LSTM ile üretim yaptı. LSTM'de 10 kelimelik pencereden sonraki kelime tahmin edildi (Adam, softmax, categorical cross entropy). Beklenen kelime gelmezse dizi kesilip o kelime eklendi ve tahmin oradan sürdü. Bu küçük veri ile Markov daha iyi sonuç verdi.
  - Bir öğrenci Mesnevi derleminde kelime tabanlı LSTM'i word2vec ve fastText ön eğitimli embedding'leriyle karşılaştırdı. Önce en sık geçen kelimeyle 10 kelimelik bir "çerçeve" cümle üretildi, sonra diğer iki kelime modelin en olası bulduğu konuma yerleştirildi. word2vec, fastText'ten daha anlamlı sonuç verdi.
  - Bir öğrenci üç kelimeyi tohum (seed) alıp fastText vektörlerinin ortalamasına en yakın kelimeyi ekleyerek metni büyüttü ve bunu LSTM ile karşılaştırdı. Sırayı modelleyen LSTM, anlamca ilişkili kelimelerde daha iyiydi; birbiriyle ilgisiz kelimelerde fastText yaklaşımı biraz daha iyi göründü.
  - T5 denemesi: İngilizce hazır bir modelle anlamlı cümleler üretildi, Türkçe eğitim verisi kurulamadı. Hoca bu veriyi kurmanın kolay bir yolunu önerdi (aşağıda).
- Memory Networks (arXiv:1410.3916)
  - YSA + memory mimarisidir; hafızaya yalnız yazar ve okur, hafızadaki bilgiyi değiştirmez. m hikayenin tutulduğu hafıza, m_{o1} ve m_{o2} cevabı üretecek hafıza parçaları, x soru, r tek kelimelik cevaptır (ekrandan, 01:20:00).
  - Benzerlik fonksiyonu: $s(x,y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$. Φ bir string'i $d \times 1$ boyutlu kelime torbası (BoW, one-hot) gösterimine çevirir; U eğitimden öğrenilen $n \times d$ boyutlu bir matristir. Boyutlar $1 \times d \cdot d \times n \cdot n \times d \cdot d \times 1 = 1 \times 1$ olduğu için sonuç tek bir sayıdır, yani iki string'in benzerliği.
  - s_O ve s_R fonksiyonlarının U matrisleri farklıdır.
  - Çıkarım: $o_1 = \arg\max_{i=1..N} s_O(x, m_i)$ soruya en çok benzeyen hafıza satırını bulur. $o_2 = \arg\max_i s_O([x, m_{o1}], m_i)$ soru ile ilk bulunan satıra birlikte en çok benzeyen

ikinci satırı bulur. $r = \arg\max_{w \in W} s_R([x, m_{o1}, m_{o2}], w)$ sözlükteki tüm kelimeler arasından cevabı seçer.
  - Eğitimde cevapla ilgili cümlelerin indisleri (bAbI'deki destekleyici cümleler) sisteme verilmelidir. Kayıp fonksiyonu gerçek m_{o1}, m_{o2} ve r ile bunların dışındaki bütün seçenekleri karşılaştırır.
- End-to-End Memory Networks (arXiv:1503.08895)
  - Eğitimde cevapla ilişkili cümlelerin işaretlenmesi gerekliliğini ortadan kaldırır.
  - $p_i = \text{Softmax}(u^\top m_i)$: m_i hafıza slotları, u sorunun embedding'i, p slotların soruyla ilişkili olma olasılıklarıdır (ekrandan, 01:25:15).
  - Cümleler A ve C embedding matrisleriyle iki ayrı uzaya taşınır. $o = \sum_i p_i c_i$ ağırlıklı toplamdır, cevap $\hat{a} = \text{Softmax}(W(o+u))$ ile sözlük boyutlu bir olasılık vektörü olarak bulunur. A, B, C, W eğitimde öğrenilir (ekrandan, 01:30:00).
  - Tek adımlı yapı tek bir hafıza bölgesine odaklanır; üç adımlı (3 hop) yapıda her adımın kendi A^k, C^k matrisleri vardır ve her adımda farklı cümleler seçilebilir.
  - Position embedding (PE): BoW toplamı kelime sırasını önemsemez. Çözüm $m_i = \sum_j l_j \cdot A x_{ij}$ (eleman bazlı çarpma), $l_{kj} = (1 - j/J) - (k/d)(1 - 2j/J)$. $d = 128$, $J = 25$ için L öğrenilmeyen sabit bir matristir; yakın konumdaki kelimelerin sütunları birbirine yakın, uzaktakiler uzaktır (ekrandan, 01:36:40).
- Veri kümelerindeki arka kapılar (tekrar)
  - Makine öğrenmesine ne yapacağını söyleyebiliriz ama nasıl yapacağını söyleyemeyiz. Nasıl yapılacağını bilseydik doğrudan kodlardık. Bu yüzden algoritma problemi hiç akla gelmeyen bir arka kapıdan çözebilir.
  - bAbI ve entailment veri kümelerinde, sistemler bazı bilgiler çıkarıldığında bile başarılı kalabiliyor. Algoritmayı kara kutu bırakmamak, veri kümesinin buna izin verip vermediğini sınamak gerekir.
- QANet (ICLR 2018) ve SQuAD
  - SQuAD'da girdi bir paragraf ve bir sorudur. Amaç her kelimenin cevabın başlangıcı ve bitişi olma olasılığını bulmaktır.
  - Her kelime GloVe tabanlı 300 + karakter tabanlı 200 = 500 boyutlu embedding ile temsil edilir. Stacked embedding encoder blokları her kelimeyi $d = 128$ boyuta indirir (ekrandan, 02:00:00).
  - Context-query attention, self-attention değildir: paragraf kelimeleri ile soru kelimeleri arasındaki ilişkiyi bulur. $S_{ij} = W_0[c; q; c \odot q]$ paragrafın i. kelimesinin sorunun j. kelimesine benzerliğidir ($n \times m$). $\bar{S}$ satır, $\bar{\bar{S}}$ sütun softmax normalizasyonudur. $A = \bar{S} \cdot Q^\top$, $B = \bar{S} \cdot \bar{\bar{S}}^\top \cdot C^\top$; çıktı $[C; A; C \odot A; C \odot B]$ ($4d \times n$).
  - Sonra stacked model encoder blokları (M_0, M_1, M_2) ve iki softmax ile başlangıç (p_1) ve bitiş (p_2) olasılıkları hesaplanır.
- Sohbet ve sanal asistan uygulamaları
  - Sohbet (chatbot) sistemlerinin ölçülebilir bir hedefi yoktur; gerçekleştirmesi çok zordur. Sanal asistanlar hedeflidir (ürün/hizmet bulma, yardım alma); hedef kullanıcının söyleyebileceklerinin uzayını kısıtlar ve sistemi kolaylaştırır.
  - ELIZA (1965, Joseph Weizenbaum) bir terapisti simüle eder. Uyan şablon varsa dönüşüm uygular, yoksa genel amaçlı cevaplardan birini verir (ekrandan, 02:14:00).
  - PARRY (Colby, 1971) paranoyak bir kişiyi taklit eder. Varsayılan bir kişiliği ve ruh hali vardır, kullanıcının söylediklerine göre değişir; yani bir durumu (state) ve geçmişi vardır. Yine kural tabanlıdır.
  - Diyalog derlemi ile cevap: derlemde kullanıcının söylediğine (q) en benzeyen ifade $t' = \arg\max_{t \in D} \frac{q^\top t}{\|q\|\|t\|}$ bulunur, t''ye verilen cevap döndürülür. Aynı yöntem sıkça sorulan sorular (SSS) için de kullanılır; en benzeyenin benzerliği eşiğin altındaysa soru insana yönlendirilir (ekrandan, 02:20:00).
  - Soft cosine: $\cos_M(X, Y) = \frac{X^\top M Y}{\sqrt{X^\top M X}\sqrt{Y^\top M Y}}$. M kelimeler arası benzerlik matrisidir: $m_{ij} = \max(0, \cos(v_i, v_j))^2$ (word2vec) ya da Levenshtein edit distance tabanlı (ekrandan, 02:21:05). M sözlük × sözlük boyutludur.
  - Seq2seq (RNN) ile diyalog üretimi; Microsoft Tay kullanıcılarla konuşmalarından öğrenmeye devam ettiği için kötü niyetli kullanıcılar ona kötü söylemler öğretti ve kısa sürede kapatıldı.

  - Görev tabanlı sanal asistanlar (goal oriented, task based, frame based): genel yapı 1977'den (GUS) beri değişmedi. Yapılabilecek işlemler, her işlem için gerekli bilgiler (slot) ve türleri, bu bilgileri almak için sorular belirlenir, bilgiler sırayla alınır (ekrandan, 02:27:10).
  - Makine öğrenmesi/NLP burada niyet (intent) tespiti ve slot doldurmada kullanılır. Örnek: "Salı sabah Ankara'ya giden uçuşlar" → niyet uçuş sorgulama, kalkış şehri (kullanıcının şehri, İstanbul), tarih, zaman, varış şehri Ankara.
  - Kullanıcıya anlaşıldığını hissettirmek gerekir: "3 büyük boy karışık pizza istiyorum" isteğine yalnız "Başka isteğiniz?" yerine "3 büyük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?" denmelidir (ekrandan, 02:31:20).

Hocanın Özellikle Vurguladığı Kısımlar

- Memory networks ve QANet gibi çalışmaların değeri kendileri değil, içlerindeki parçalardır
  - Hoca bu çalışmaların "reasoning'i çözdük" ya da "bilgisayar artık paragrafı anlıyor" gibi sonuçlara götürmediğini söyledi. Ama position embedding ve attention gibi parçalar sonraki çalışmalara (Transformer/BERT) esin kaynağı oldu; derste ayrıntılı anlatılmalarının sebebi bu.
- Derin öğrenme mimarilerindeki "ad hoc" yapılar
  - QANet'teki A ve B gibi yapılar mantıklı ama gelişigüzel kurulmuş yapılardır. Bir veri kümesinde en iyi çıkan yapının başka bir veri kümesinde de en iyi olacağı söylenemez.
- Arka kapılar
  - Ne yapılacağını söyleriz, nasıl yapılacağını söyleyemeyiz; algoritma problemi beklenmedik ipuçlarıyla çözebilir. Veri kümesi buna izin veriyor mu, test edilmelidir.
- Kullanıcıdan öğrenmenin riski
  - Microsoft Tay örneği: kötü niyetli kullanıcılar sistemi yanlış yönlendirmeye her zaman çalışacaktır, özellikle sistemler karar vermeye başladıkça.
- Görev tabanlı sistemlerin yapısı
  - Slot-soru yapısı 1977'den beri aynıdır. Ticari uygulamalarda yanlış işlem çok pahalı olduğu için sistemler kural setleriyle sıkı tutulur; doğal dil işleme serbest bırakılmaz.

Kısa Tekrar Notları

- Markov zinciri ile üretimde rastgelelik, geçiş olasılıklarına göre ağırlıklı rastgele seçimden (weighted choice) gelir; aynı başlangıç kelimesiyle farklı cümleler üretilebilir.
- T5 ile üç kelimeden cümle üretimi için eğitim verisi: rastgele bir cümleden rastgele üç kelime al → girdi dizisi; cümlenin kendisi → çıktı dizisi (seq2seq).
- Memory networks: o_1 soruya, o_2 soru + m_{o1}'e en benzeyen hafıza satırı; r sözlükten seçilen tek kelime. Eğitimde destekleyici cümle indisleri gerekir.
- argmax, fonksiyonu en büyük yapan argümanı (burada indisi ya da kelimeyi) döndürür; argmin en küçük yapanı.
- End-to-end memory networks: $p_i = \text{Softmax}(u^\top m_i)$, $o = \sum p_i c_i$, $\hat{a} = \text{Softmax}(W(o+u))$. Destekleyici cümle işareti gerekmez.
- BoW toplamında kelime sırası kaybolur (3+5 = 5+3). Position embedding bu sorunu çözer; E2E memory networks'te sabit bir L matrisiyle yapılır.
- QANet: kelime 500 boyutlu girdi (300 GloVe + 200 karakter), 128 boyutlu kodlama; context-query attention; başlangıç ve bitiş olasılıkları.
- ELIZA: şablon + dönüşüm, geçmişi yok. PARRY: durum (state) ve kişilik var.
- Diyalog derleminde en benzer ifadeyi kosinüsle bul, ona verilen cevabı döndür. Anlamsal benzerlik için soft cosine.
- Görev tabanlı asistan: niyet tespiti + slot doldurma + eksik slot için soru + anlaşıldığını hissettiren teyit.

Detaylı Açıklamalar

- Ödev sunumlarından çıkan dersler
  - Markov zinciri küçük derlemde ikili-üçlü kelime gruplarını derlemden neredeyse aynen aldığı için yerel olarak anlamlı, bütün olarak çoğunlukla anlamsız cümleler üretti.

LSTM'ler az veri ve kısa eğitim yüzünden genelde daha kötü kaldı.
  - Hoca, karakter tabanlı LSTM ile kelime tabanlı Markov'un karşılaştırılmasında iki farkın (karakter/kelime ve yöntem) birden değiştiğini, bu yüzden karşılaştırmanın tam adil olmadığını belirtti.
  - "Çerçeve cümle + araya kelime yerleştirme" yönteminde çerçeve cümlenin kendisi iyi olmalıdır, çünkü araya sokulan her kelime cümleyi bir miktar bozar. Daha büyük pencere ve daha büyük derlemle daha iyi sonuç beklenir.
  - fastText ortalama vektörüyle metin büyütme sırayı hiç modellemez; LSTM ardışıklığı içine aldığı için genelde daha başarılı olması beklenir.
- Memory networks mekanizması
  - Test zamanında önce o_1, sonra o_2 bulunur, sonra x, m_{o1}, m_{o2} ile cevap sözlükten seçilir. Bu yapı cevaba iki cümleyle ulaşılan görev için kurulmuştur; daha fazla cümle gerekirse o_3, o_4 diye sürer.
  - Hoca U'nun anlamsal bir benzerlik kattığını anlattı. U olmasa iki one-hot vektörün iç çarpımı yalnız ortak kelimeleri yakalar. U ile çarpmak word2vec'teki gibi one-hot vektörü bir embedding matrisiyle çarpmaktır.
  - Kayıp fonksiyonu üç satırdan oluşur: m_{o1}, m_{o2} ve r için gerçek seçenek, gerçek olmayan bütün seçeneklerle ($\bar{f}$, $\bar{r}$) karşılaştırılır. Fonksiyondaki ek bir terim değerlerin sıfıra gitmesini engeller.
- End-to-end yapının bir avantajı
  - p vektörü tek bir 1 ve geri kalanı 0 değildir (örneğin 0,5 ve 0,4 gibi değerler olabilir). Bu yüzden tek adımlı yapı bile birden fazla hafıza slotunu kullanabilir. Üç adımlı yapı bunu daha açık hâle getirir.
  - Makaledeki görev tablolarında (20 görev) her hop'ta hangi cümlelerin aktif olduğu gösterilir. Bazı sorularda hop'lar aynı cümlede kalır, bazılarında farklı cümlelere geçer. Destekleyici cümleler verilmeden de doğru cümlelerin bulunabildiği görülür.
- Sanal asistanlarda dikkat edilecekler
  - Sesli kontrol (evde, arabada) ve yüzlerce seçenek içinden seçim yapmak gerektiğinde sanal asistan klavyeye göre avantajlıdır.
  - Kısmen anlaşılan ifadede anlaşılanı söyleyip eksik olanı sormak gerekir ("Salı günü nereye gitmek istiyorsunuz?"). Varsayım yapıldıysa ("İstanbul'dan Ankara'ya hangi tarih için uçuş arıyorsunuz?") kullanıcının düzeltmesi de anlaşılabilmelidir.
  - Ticari araçlar: Dialogflow (Google), Amazon'un ve IBM Watson'ın araçları. Türkçe için bu alanda büyük bir boşluk vardır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.